

ar1 Veridi Logistics — Last Mile Delivery Audit

Client: Veridi Logistics (Global E-Commerce Aggregator)

Analyst: *Justine Umutoni*

Dataset: [Olist Brazilian E-Commerce Dataset](#)

Objective

Audit delivery performance to determine whether:

1. Estimated delivery dates are systematically inaccurate
 2. Specific regions are disproportionately affected
 3. Late deliveries are the root cause of negative customer reviews
-

Notebook Structure

Section	Description
0	Setup & Dependencies
1	Data Loading & Schema Building (Story 1)
2	Delay Calculator (Story 2)
3	Geographic Heatmap (Story 3)
4	Sentiment Correlation (Story 4)
5	Bonus: Category Analysis (Translation Challenge)
6	Candidate's Choice: Delivery Promise Reliability Score
7	Export for Dashboard

ar1 Section 0 — Setup & Dependencies

Reproducibility note: This notebook downloads the dataset directly from Kaggle using the `kaggle` API. Alternatively, place all CSV files in the **same folder as this notebook** and skip the download cell.

To use Kaggle API:

1. Go to <https://www.kaggle.com/settings> → API → Create New Token
2. Upload `kaggle.json` when prompted (Colab) or place it at `~/.kaggle/kaggle.json` (local)

```
# — Install dependencies —————
!pip install -q kaggle plotly kaleido folium geopandas matplotlib seaborn pandas numpy
```

```
===== 55.6/55.6 kB 2.3 MB/s eta 0:00:00
===== 52.6/52.6 kB 4.3 MB/s eta 0:00:00
```

```
# Kaggle Authentication (Colab)

import os
try:
    from google.colab import files
    IN_COLAB = True
    print("Running in Colab. Upload your kaggle.json file when prompted.")
    files.upload() # upload kaggle.json
    os.makedirs('/root/.kaggle', exist_ok=True)
    !cp kaggle.json /root/.kaggle/
    !chmod 600 /root/.kaggle/kaggle.json
except ModuleNotFoundError:
    IN_COLAB = False
    print("Not running in Colab – ensure kaggle.json is at ~/.kaggle/kaggle.json")
```

Running in Colab. Upload your kaggle.json file when prompted.

No files selected. Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

```
# Download Dataset from Kaggle
```

```
DATA_DIR = "data"

if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)
    !kaggle datasets download -d olistbr/brazilian-ecommerce -p {DATA_DIR} --unzip
    print(f"Dataset downloaded to ./{DATA_DIR}/")
else:
    print(f"Data directory already exists. Skipping download.")

# List available files
print("\nAvailable CSV files:")
for f in sorted(os.listdir(DATA_DIR)):
    print(f"  {f}")
```

Dataset URL: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

License(s): CC-BY-NC-SA-4.0

Downloading brazilian-ecommerce.zip to data

100% 42.6M/42.6M [00:00<00:00, 145MB/s]

Dataset downloaded to ./data/

Available CSV files:

```
olist_customers_dataset.csv
olist_geolocation_dataset.csv
olist_order_items_dataset.csv
olist_order_payments_dataset.csv
olist_order_reviews_dataset.csv
olist_orders_dataset.csv
olist_products_dataset.csv
olist_sellers_dataset.csv
product_category_name_translation.csv
```

```
# Imports Libraries
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns
```

```

import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import warnings

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 50)
pd.set_option('display.float_format', '{:.2f}'.format)

# Color Palette (Veridi Brand)

NAVY    = '#0A2342'
TEAL    = '#1B9AAA'
AMBER   = '#F4A261'
RED     = '#E63946'
GREEN   = '#2DC653'
LIGHT   = '#F8F9FA'

PALETTE = [TEAL, AMBER, RED, GREEN, NAVY]

```

ari Section 1 — Schema Builder (User Story 1)

Goal: Join Orders → Reviews → Customers into one master DataFrame.

```

# 1.1 Load Raw CSVs

DATA = DATA_DIR # alias

orders = pd.read_csv(f"{DATA}/olist_orders_dataset.csv", parse_dates=['order_purchase_timestamp',
                                                                    'order_approved_at',
                                                                    'order_delivered_carrier_date',
                                                                    'order_delivered_customer_date',
                                                                    'order_estimated_delivery_date'])

```

```
reviews = pd.read_csv(f"{DATA}/olist_order_reviews_dataset.csv", parse_dates=['review_creation_date',
                                                                              'review_answer_timestamp'])

customers = pd.read_csv(f"{DATA}/olist_customers_dataset.csv")
products = pd.read_csv(f"{DATA}/olist_products_dataset.csv")
order_items = pd.read_csv(f"{DATA}/olist_order_items_dataset.csv")
translations = pd.read_csv(f"{DATA}/product_category_name_translation.csv")

print(f"orders      : {orders.shape}")
print(f"reviews      : {reviews.shape}")
print(f"customers      : {customers.shape}")
print(f"products        : {products.shape}")
print(f"order_items     : {order_items.shape}")
print(f"translations: {translations.shape}")
```

```
orders      : (99441, 8)
reviews      : (99224, 7)
customers      : (99441, 5)
products      : (32951, 9)
order_items   : (112650, 7)
translations: (71, 2)
```

1.2 Quick Schema Peek

```
print("=== ORDERS ===")
display(orders.head(3))
print("\n=== REVIEWS ===")
display(reviews.head(3))
print("\n=== CUSTOMERS ===")
display(customers.head(3))
```

=== ORDERS ===

	order_id	customer_id	order_status	order_purchase_timestamp
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49

```
=== REVIEWS ===
```

	review_id	order_id	review_score	review_comment_title	re
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NaN	
1	80e641a11e56f04c1ad469d5645fdfde	a548910a1c6147796b98fdf73dbeba33	5	NaN	
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5	NaN	

```
=== CUSTOMERS ===
```

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_ci
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	fran
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790	sao bernar do cam
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151	sao pa

```
# 1.3 Cardinality Check (Prevent Row Duplication) -
```

```
# Reviews: can have multiple reviews per order?
```

```
dup_reviews = reviews.groupby('order_id')['review_id'].count()
print("Reviews per order distribution:")
print(dup_reviews.value_counts().to_string())
print(f"\nOrders with >1 review: {(dup_reviews > 1).sum()}")

reviews_deduped = (
    reviews
    .sort_values('review_answer_timestamp', ascending=False)
    .drop_duplicates(subset='order_id', keep='first')
)
print(f"\nReviews after dedup: {reviews_deduped.shape[0]} (was {reviews.shape[0]})")
```

```
Reviews per order distribution:
```

```
review_id
1      98126
2       543
```

3

4

Orders with >1 review: 547

Reviews after dedup: 98673 (was 99224)

1.4 Build Master Dataset

```
master = (
    orders
    # Join reviews (left – keep all orders, even unreviewed)
    .merge(reviews_deduped[['order_id', 'review_score', 'review_comment_title',
                           'review_comment_message', 'review_creation_date']],
          on='order_id', how='left')

    # Join customers
    .merge(customers[['customer_id', 'customer_city', 'customer_state',
                     'customer_zip_code_prefix']],
          on='customer_id', how='left')
)

# row count must equal orders
assert master.shape[0] == orders.shape[0], (
    f"Row count mismatch! Expected {orders.shape[0]}, got {master.shape[0]}. Check for duplicate keys."
)
print(f"Master dataset shape: {master.shape} ✓ (matches orders row count)")
display(master.head(3))
```

Master dataset shape: (99441, 15) ✓ (matches orders row count)

	order_id	customer_id	order_status	order_purchase_timestamp
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49

ar Section 2 — Delay Calculator (User Story 2)

Goal: Compute `delivery_delay_days` = Estimated Date – Actual Delivery Date.

Sign convention: Positive = arrived early (good). Negative = arrived late (bad).

Classification: `>= 0` → On Time | `-1 to -5` → Late | `< -5` → Super Late.

Alternative convention `actual - estimated` (positive = late) is equally valid — the key is consistency.

```
# 2.1 Inspect Order Statuses
```

```
print("Order status distribution:")
print(master['order_status'].value_counts().to_string())
```

```
Order status distribution:
order_status
delivered      96478
shipped        1107
canceled         625
unavailable     609
invoiced        314
processing      301
created          5
approved         2
```

```
# 2.2 Filter to Delivered Orders Only
```

```
EXCLUDED_STATUSES = ['canceled', 'unavailable']
delivered = master[master['order_status'] != 'delivered'].copy()
print(f"Delivered orders: {len(delivered):,} (excluded {len(master) - len(delivered):,} non-delivered)")
```

```
# Drop rows where either date is null
```

```
delivered = delivered.dropna(subset=['order_estimated_delivery_date', 'order_delivered_customer_date'])
```

```
print(f"After dropping null dates: {len(delivered):,}")
```

```
Delivered orders: 96,478 (excluded 2,963 non-delivered)
After dropping null dates: 96,470
```

```
# — 2.3 Calculate Days_Difference —————
# Positive = delivered BEFORE estimate (early / on time – good)
# Zero     = delivered exactly on estimate
# Negative = delivered AFTER estimate (late – bad)

delivered['Days_Difference'] = (
    (delivered['order_estimated_delivery_date'] - delivered['order_delivered_customer_date'])
    .dt.days
)

print("Days_Difference summary:")
print(delivered['Days_Difference'].describe())
```

```
Days_Difference summary:
count    96470.00
mean       10.88
std        10.18
min       -189.00
25%         6.00
50%        11.00
75%        16.00
max       146.00
Name: Days_Difference, dtype: float64
```

```
# 2.4 Classify Delivery Status
```

```
def classify_delivery(days):
    if days >= 0:
        return 'On Time'
    elif days >= -5:
        return 'Late'
    else:
        return 'Super Late'
```

```
delivered['Delivery_Status'] = delivered['Days_Difference'].apply(classify_delivery)

# Summary
status_counts = delivered['Delivery_Status'].value_counts()
status_pct = delivered['Delivery_Status'].value_counts(normalize=True) * 100
summary = pd.DataFrame({'Count': status_counts, 'Percentage': status_pct.round(1)})
print("Delivery Status Summary:")
display(summary)
```

Delivery Status Summary:

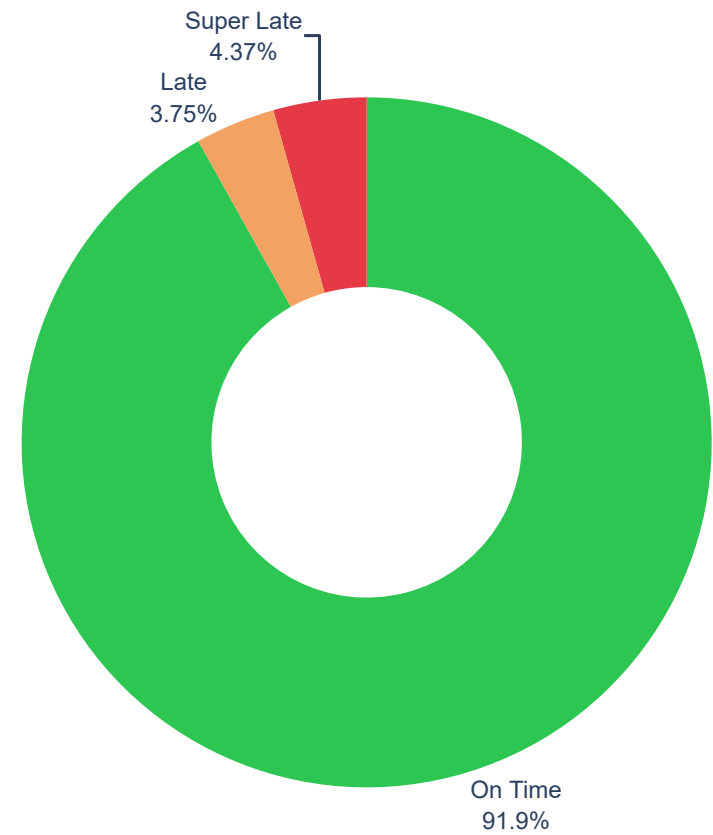
	Count	Percentage
Delivery_Status		
On Time	88644	91.90
Super Late	4211	4.40
Late	3615	3.70

2.5 Visualise Delivery Status Distribution

```
fig = px.pie(
    names=summary.index,
    values=summary['Count'],
    title='Overall Delivery Status Distribution',
    color=summary.index,
    color_discrete_map={
        'On Time': GREEN,
        'Late': AMBER,
        'Super Late': RED
    },
    hole=0.45
)
fig.update_traces(textposition='outside', textinfo='percent+label')
fig.update_layout(
    font_family='Arial',
    _ _ _ _ _
```

```
    title_font_size=18,  
    showlegend=True  
)  
fig.show()
```

Overall Delivery Status Distribution

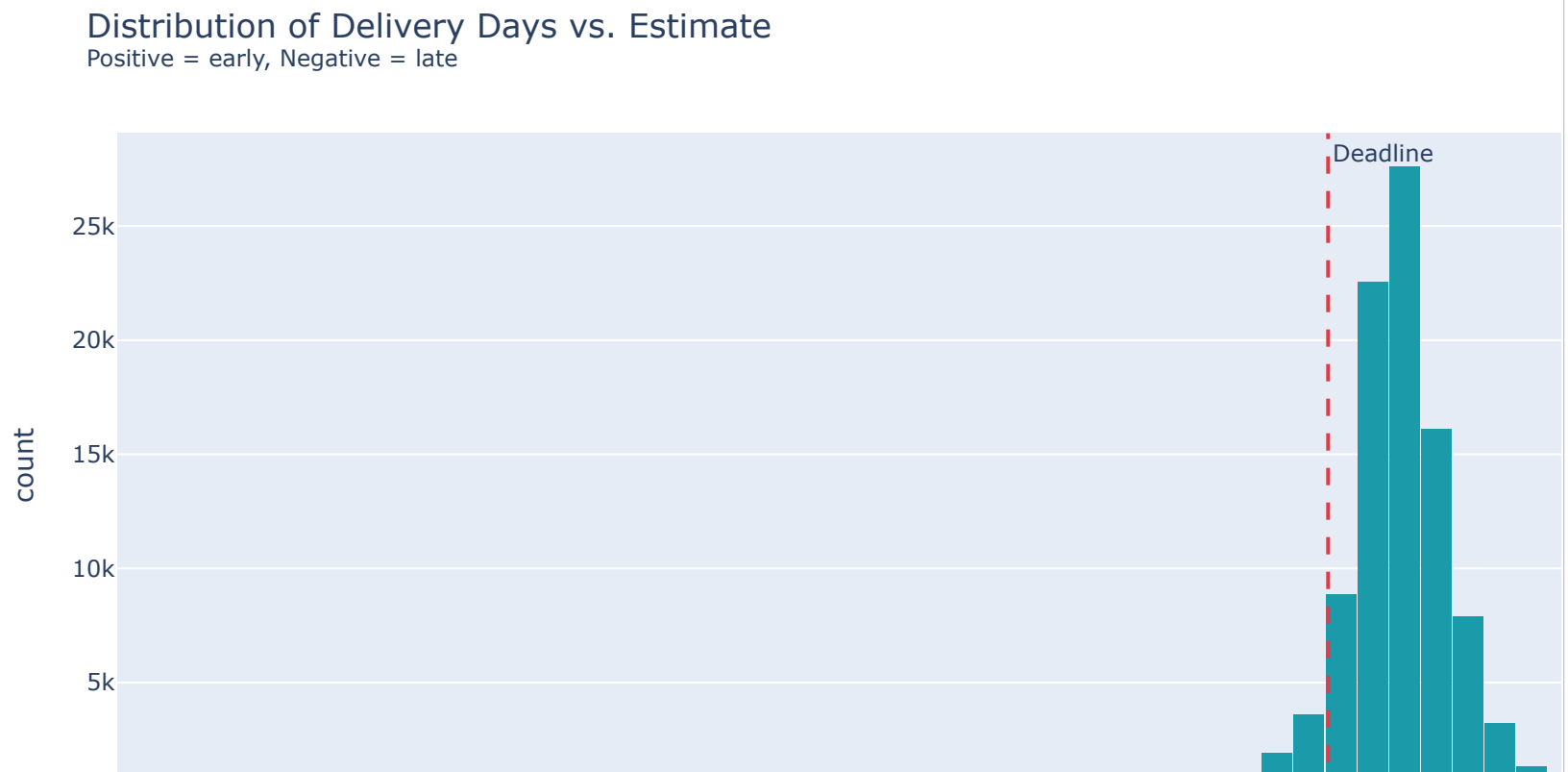


#2.6 Distribution of Days_Difference

```
fig = px.histogram(  
    delivered,
```

```
x='Days_Difference',
nbins=80,
color_discrete_sequence=[TEAL],
title='Distribution of Delivery Days vs. Estimate<br><sup>Positive = early, Negative = late</sup>',
labels={'Days_Difference': 'Days (Estimate - Actual)'}
)
fig.add_vline(x=0, line_dash='dash', line_color=RED, annotation_text='Deadline', annotation_position='top r
fig.update_layout(bargap=0.05)
fig.show()

early = (delivered['Days_Difference'] > 0).sum()
late = (delivered['Days_Difference'] < 0).sum()
print(f"Orders delivered early / on time : {early:,,} ({early/len(delivered)*100:.1f}%)")
print(f"Orders delivered late : {late:,,} ({late/len(delivered)*100:.1f}%")
```





Orders delivered early / on time : 87,182 (90.4%)
 Orders delivered late : 7,826 (8.1%)

ar1 Section 3 — Geographic Heatmap (User Story 3)

Goal: Identify which Brazilian states have the highest percentage of late deliveries.

3.1 Aggregate by State

```
state_agg = (
    delivered
    .groupby('customer_state')
    .agg(
        Total_Orders      = ('order_id', 'count'),
        Late_Orders       = ('Delivery_Status', lambda x: (x != 'On Time').sum()),
        Avg_Delay_Days    = ('Days_Difference', 'mean'),
        Avg_Review_Score = ('review_score', 'mean')
    )
    .reset_index()
)

state_agg['Late_Pct'] = (state_agg['Late_Orders'] / state_agg['Total_Orders'] * 100).round(1)
state_agg = state_agg.sort_values('Late_Pct', ascending=False)

print("Top 10 States by Late Delivery %:")
display(state_agg.head(10))
```

Top 10 States by Late Delivery %:

customer_state	Total_Orders	Late_Orders	Avg_Delay_Days	Avg_Review_Score	Late_Pct
----------------	--------------	-------------	----------------	------------------	----------

1	AL	397	95	7.71	3.85	23.90
9	MA	717	141	8.57	3.83	19.70
16	PI	476	76	10.31	3.99	16.00
5	CE	1279	196	9.80	3.94	15.30
24	SE	335	51	9.02	3.91	15.20
4	BA	3256	457	9.79	3.93	14.00
18	RJ	12350	1664	10.76	3.97	13.50
26	TO	274	35	11.13	4.15	12.80
13	PA	946	117	13.07	3.91	12.40
21	RR	41	5	16.29	3.90	12.20

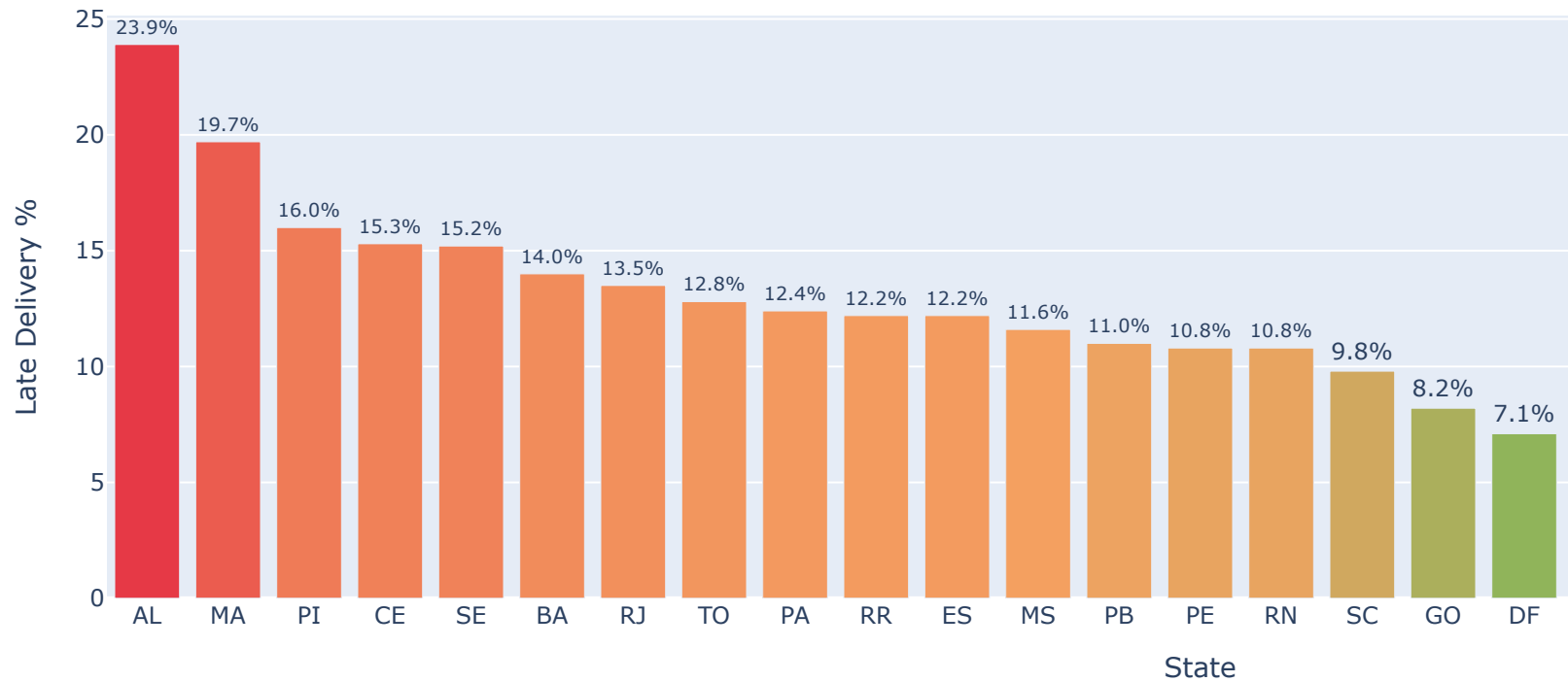
3.2 Bar Chart: Late % by State

```

fig = px.bar(
    state_agg,
    x='customer_state',
    y='Late_Pct',
    color='Late_Pct',
    color_continuous_scale=[[0, GREEN], [0.4, AMBER], [1, RED]],
    title='Late Delivery Rate by Brazilian State (%)',
    labels={'customer_state': 'State', 'Late_Pct': 'Late Delivery %'},
    text='Late_Pct'
)
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(
    xaxis={'categoryorder': 'total descending'},
    coloraxis_showscale=False,
    height=480
)
fig.show()

```

Late Delivery Rate by Brazilian State (%)



```
# 3.3 Choropleth Map (Brazil GeoJSON)
# Load Brazil states GeoJSON (sourced from public domain)

import requests, json

GEOJSON_URL = (
    "https://raw.githubusercontent.com/codeforamerica/click_that_hood/"
    "master/public/data/brazil-states.geojson"
)

try:
```



```
response = requests.get(GEOJSON_URL, timeout=10)
brazil_geo = response.json()

# Build state-code → feature mapping
# The GeoJSON uses full state names; map to 2-letter codes used in our data

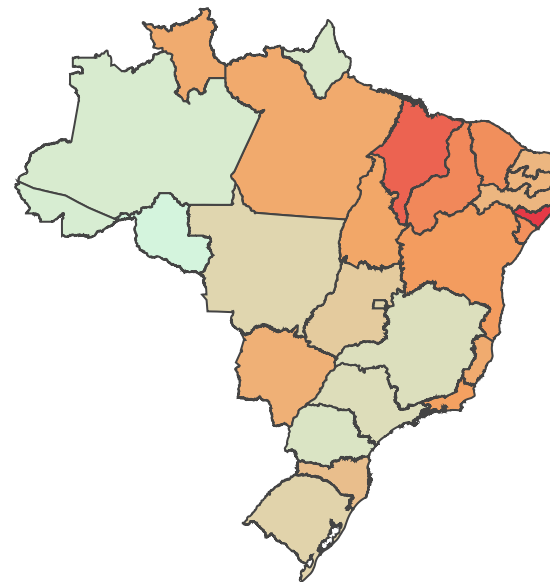
STATE_ABBREV = {
    'Acre': 'AC', 'Alagoas': 'AL', 'Amapá': 'AP', 'Amazonas': 'AM',
    'Bahia': 'BA', 'Ceará': 'CE', 'Distrito Federal': 'DF',
    'Espírito Santo': 'ES', 'Goiás': 'GO', 'Maranhão': 'MA',
    'Mato Grosso': 'MT', 'Mato Grosso do Sul': 'MS', 'Minas Gerais': 'MG',
    'Pará': 'PA', 'Paraíba': 'PB', 'Paraná': 'PR', 'Pernambuco': 'PE',
    'Piauí': 'PI', 'Rio de Janeiro': 'RJ', 'Rio Grande do Norte': 'RN',
    'Rio Grande do Sul': 'RS', 'Rondônia': 'RO', 'Roraima': 'RR',
    'Santa Catarina': 'SC', 'São Paulo': 'SP', 'Sergipe': 'SE',
    'Tocantins': 'TO'
}

for feature in brazil_geo['features']:
    name = feature['properties'].get('name', '')
    feature['properties']['sigla'] = STATE_ABBREV.get(name, name)

fig = px.choropleth(
    state_agg,
    geojson=brazil_geo,
    locations='customer_state',
    featureidkey='properties.sigla',
    color='Late_Pct',
    color_continuous_scale=[[0, '#d4f4dd'], [0.5, AMBER], [1, RED]],
    scope='south america',
    title='Late Delivery Rate by State – Brazil Heatmap',
    labels={'Late_Pct': 'Late %', 'customer_state': 'State'},
    hover_data=['Total_Orders', 'Avg_Delay_Days', 'Avg_Review_Score']
)
fig.update_geos(fitbounds='locations', visible=False)
fig.update_layout(height=600)
fig.show()
```

```
except Exception as e:  
    print(f"Could not load GeoJSON (network issue?): {e}")  
    print("Falling back to bar chart – see previous cell.")
```

Late Delivery Rate by State — Brazil Heatmap



3 4 Remote vs. Core States Insight

```

# Remote vs. Core States Insight
# States geographically distant from São Paulo (main distribution hub)

REMOTE_STATES = ['AM', 'PA', 'RR', 'AP', 'AC', 'RO', 'TO', 'MA', 'PI', 'RN', 'PB', 'AL']
CORE_STATES   = ['SP', 'RJ', 'MG', 'PR', 'SC', 'RS', 'ES', 'GO', 'DF', 'MS', 'MT']

delivered['Region_Type'] = delivered['customer_state'].apply(
    lambda s: 'Remote' if s in REMOTE_STATES else ('Core' if s in CORE_STATES else 'Other')
)

region_summary = (
    delivered
    .groupby('Region_Type')
    .agg(
        Orders      = ('order_id', 'count'),
        Late_Pct     = ('Delivery_Status', lambda x: round((x != 'On Time').mean() * 100, 1)),
        Avg_Score    = ('review_score', lambda x: round(x.mean(), 2))
    )
    .reset_index()
)

print("Remote vs. Core State Performance:")
display(region_summary)

fig = px.bar(
    region_summary,
    x='Region_Type', y='Late_Pct',
    color='Region_Type',
    color_discrete_map={'Remote': RED, 'Core': TEAL, 'Other': AMBER},
    text='Late_Pct',
    title='Late Delivery % – Remote vs. Core States',
    labels={'Late_Pct': 'Late Delivery %', 'Region_Type': 'Region Type'})
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.show()

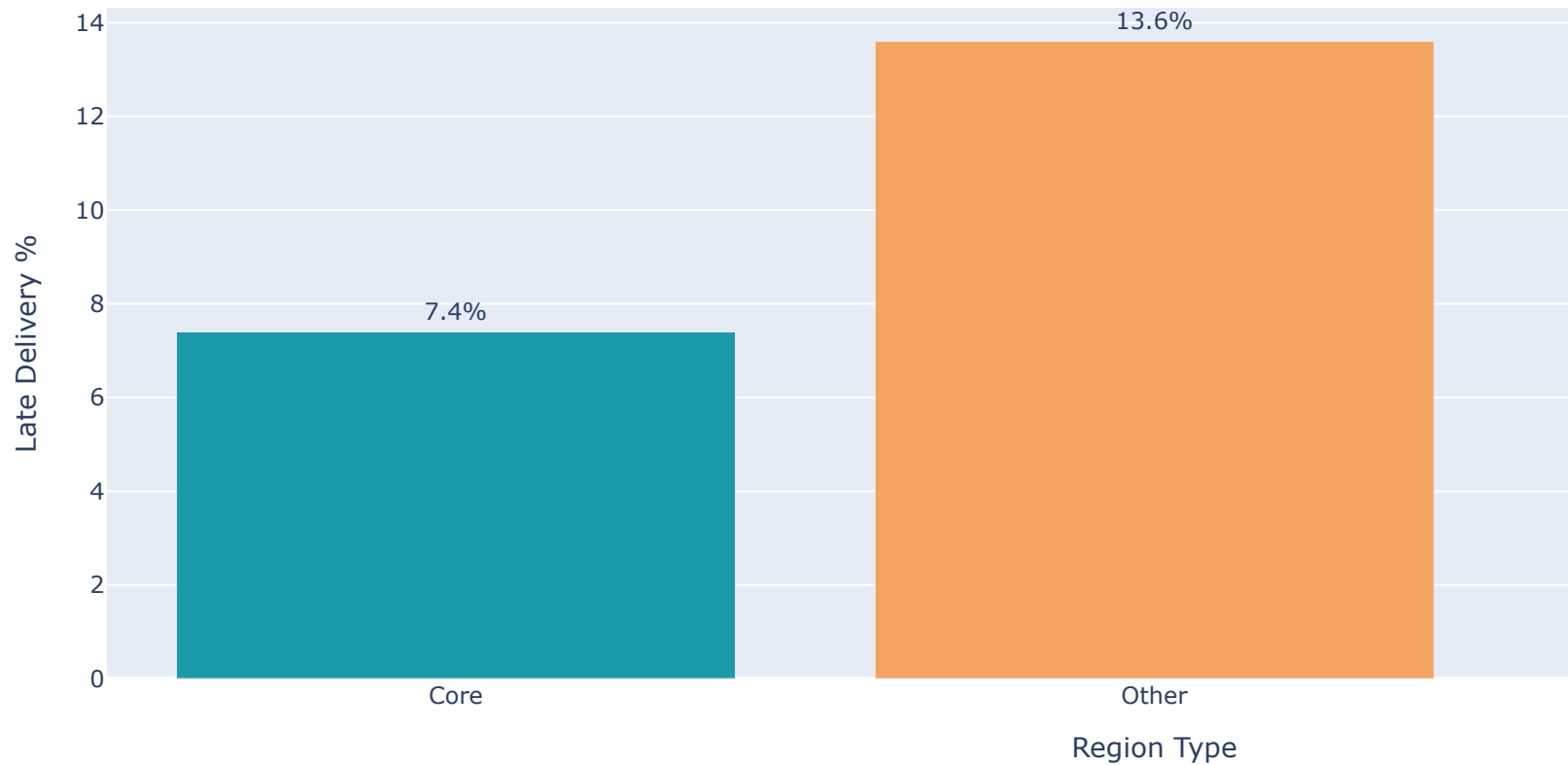
```

Remote vs. Core State Performance:

	Region_Type	Orders	Late_Pct	Avg_Score
0	Core	85630	7.40	4.18

	State	Count	Rate	Rate
1	Other	6463	13.60	3.97
2	Remote	4377	13.60	4.00

Late Delivery % — Remote vs. Core States



an Section 4 — Sentiment Correlation (User Story 4)

Goal: Prove that late deliveries cause lower review scores

Goal: Prove that late deliveries cause lower review scores.

```
# 4.1 Review Score by Delivery Status
```

```
score_by_status = (
    delivered
    .dropna(subset=['review_score'])
    .groupby('Delivery_Status')['review_score']
    .agg(['mean', 'median', 'count'])
    .round(2)
    .reset_index()
    .rename(columns={'mean': 'Avg_Score', 'median': 'Median_Score', 'count': 'n'})
)
print("Review Score by Delivery Status:")
display(score_by_status)
```

Review Score by Delivery Status:

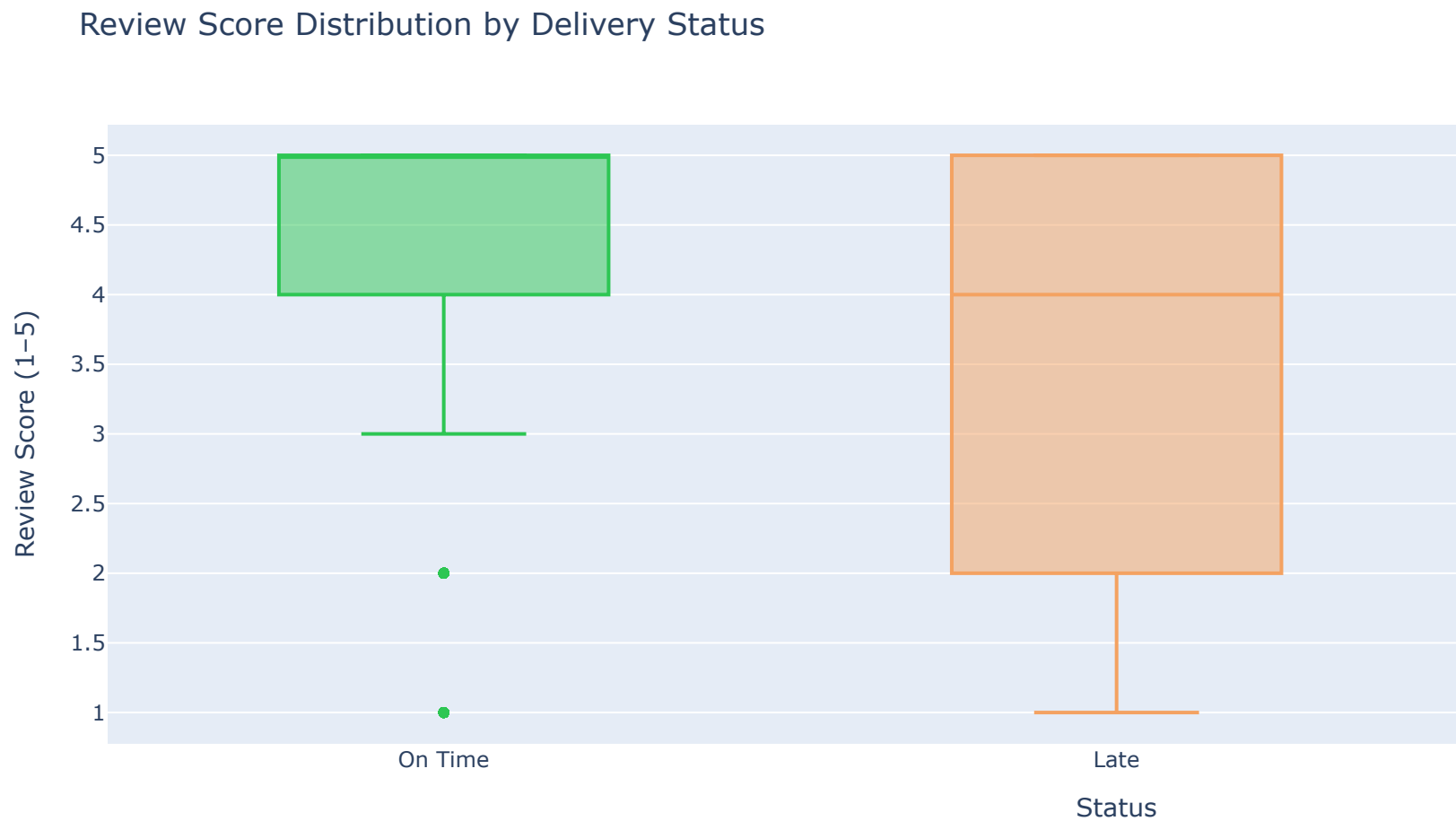
	Delivery_Status	Avg_Score	Median_Score	n
0	Late	3.46	4.00	3568
1	On Time	4.29	5.00	88163
2	Super Late	1.78	1.00	4093

```
# Order for plotting
```

```
STATUS_ORDER = ['On Time', 'Late', 'Super Late']
```

```
fig = px.box(
    delivered.dropna(subset=['review_score']),
    x='Delivery_Status',
    y='review_score',
    color='Delivery_Status',
    category_orders={'Delivery_Status': STATUS_ORDER},
    color_discrete_map={'On Time': GREEN, 'Late': AMBER, 'Super Late': RED},
    title='Review Score Distribution by Delivery Status',
    labels={'review_score': 'Review Score (1-5)', 'Delivery_Status': 'Status'})
```

```
labels=[ 'review_score' : 'review_score (1-5)', 'delivery_status' : 'status'],  
points='outliers'  
)  
fig.update_layout(showlegend=False)  
fig.show()
```



```
# 4.2 Scatter plot: Delay Days vs. Review Score  
# Bin by delay bucket for clarity
```

```

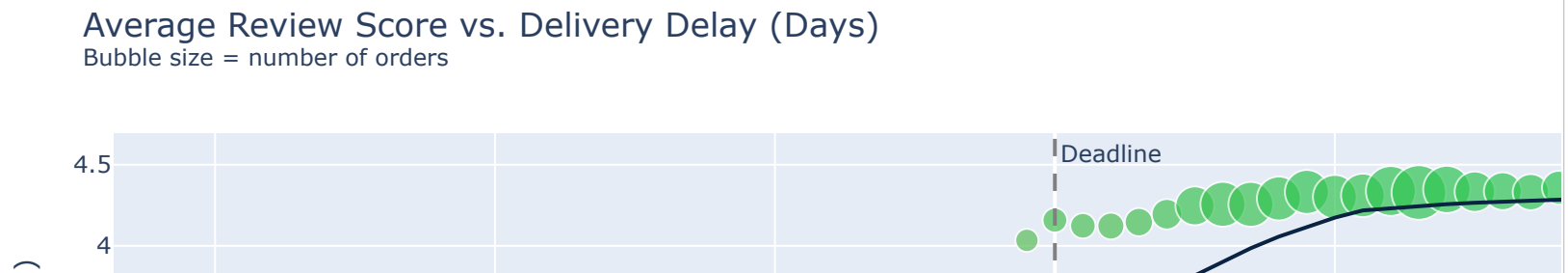
plot_df = delivered.dropna(subset=['review_score']).copy()

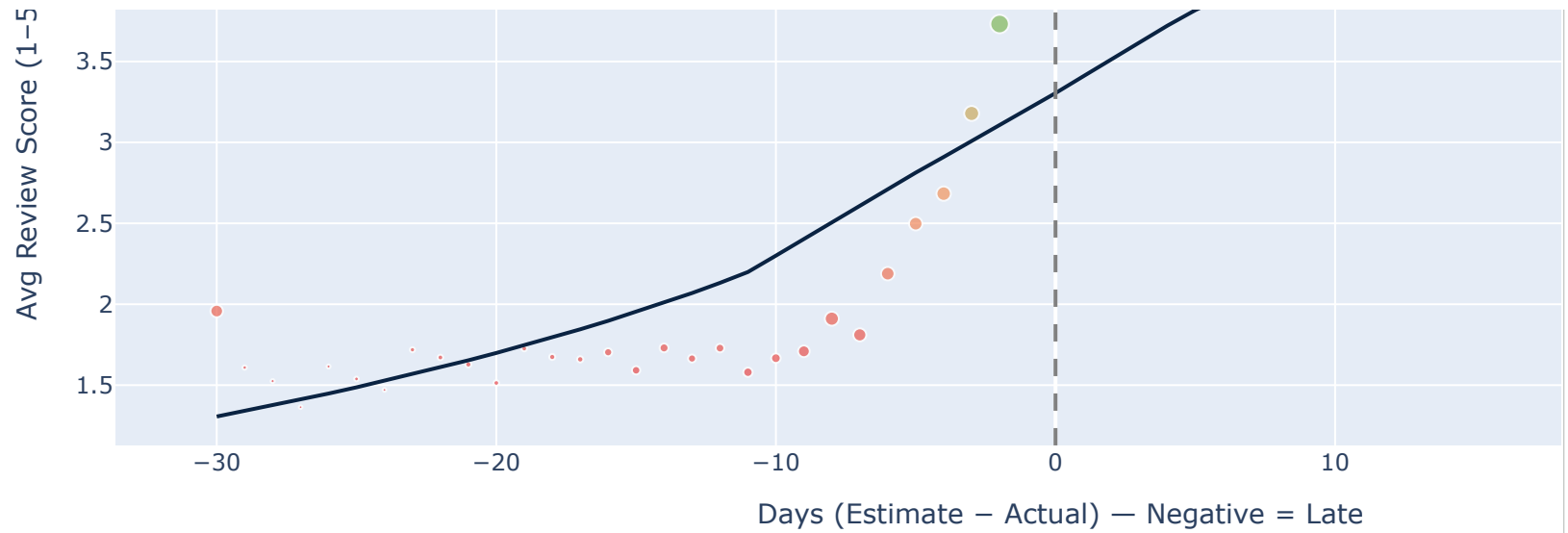
# Bucket delay into integer bins -30 to +30
plot_df['Delay_Bucket'] = plot_df['Days_Difference'].clip(-30, 30).astype(int)

scatter_agg = (
    plot_df
    .groupby('Delay_Bucket')
    .agg(Avg_Score=('review_score', 'mean'), n=('review_score', 'count'))
    .reset_index()
)

fig = px.scatter(
    scatter_agg,
    x='Delay_Bucket',
    y='Avg_Score',
    size='n',
    color='Avg_Score',
    color_continuous_scale=[[0, RED], [0.5, AMBER], [1, GREEN]],
    title='Average Review Score vs. Delivery Delay (Days)<br><sup>Bubble size = number of orders</sup>',
    labels={'Delay_Bucket': 'Days (Estimate - Actual) - Negative = Late',
           'Avg_Score': 'Avg Review Score (1-5)'},
    trendline='lowess',
    trendline_color_override=NAVY
)
fig.add_vline(x=0, line_dash='dash', line_color='grey', annotation_text='Deadline')
fig.update_layout(height=480)
fig.show()

```





4.3 Review Score Distribution (Heatmap: Status × Score)

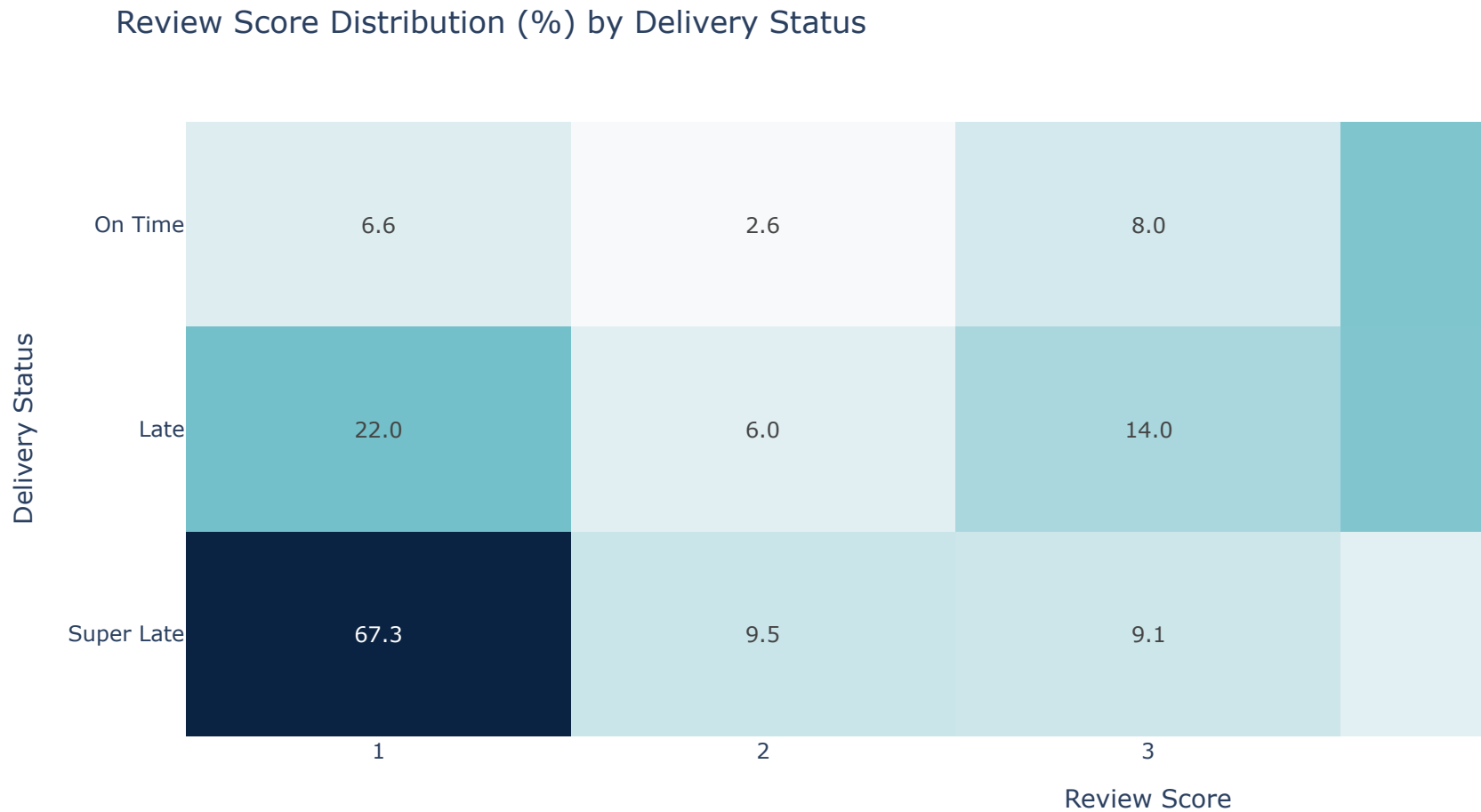
```
heat_df = (
    delivered
    .dropna(subset=['review_score'])
    .groupby(['Delivery_Status', 'review_score'])
    .size()
    .reset_index(name='Count')
    .pivot(index='Delivery_Status', columns='review_score', values='Count')
    .fillna(0)
    .loc[STATUS_ORDER]
)

# Normalise rows to % to make comparison fair
heat_pct = heat_df.div(heat_df.sum(axis=1), axis=0) * 100

fig = px.imshow(
    heat_pct,
    text_auto='.1f',
    color_continuous_scale=[[0, LIGHT], [0.5, TEAL], [1, NAVY]],
```



```
title='Review Score Distribution (%) by Delivery Status',  
labels={'x': 'Review Score', 'y': 'Delivery Status', 'color': '%'},  
aspect='auto'  
)  
fig.show()
```



4.4 Statistical Significance (Kruskal-Wallis Test)

```
from scipy import stats
```

```

ontime_scores = delivered[delivered['Delivery_Status'] == 'On Time']['review_score'].dropna()
late_scores   = delivered[delivered['Delivery_Status'] == 'Late']['review_score'].dropna()
slat_scores   = delivered[delivered['Delivery_Status'] == 'Super Late']['review_score'].dropna()

stat, pvalue = stats.kruskal(ontime_scores, late_scores, slat_scores)

print(f"Kruskal-Wallis H-statistic : {stat:.2f}")
print(f"p-value                   : {pvalue:.2e}")
if pvalue < 0.05:
    print(" Result is STATISTICALLY SIGNIFICANT (p < 0.05)")
    print("   → Late deliveries DO cause significantly lower review scores.")
else:
    print(" Result is NOT statistically significant.")

```

```

Kruskal-Wallis H-statistic : 9886.13
p-value                   : 0.00e+00
Result is STATISTICALLY SIGNIFICANT (p < 0.05)
→ Late deliveries DO cause significantly lower review scores.

```

ar1 Section 5 — Bonus: Category Analysis with Portuguese→English Translation

Goal: Determine if shipping difficulty varies by product category.

```

# 5.1 Join Products → Order Items → Master

# One order can have multiple items
# Add English translation to products

products_en = products.merge(translations, on='product_category_name', how='left')

# Take the first item per order

first_item = (
    order items

```

```

        .sort_values('order_item_id')
        .drop_duplicates(subset='order_id', keep='first')
        [['order_id', 'product_id']]
    )

    # Join product category

    first_item = first_item.merge(
        products_en[['product_id', 'product_category_name', 'product_category_name_english']],
        on='product_id', how='left'
    )

    # Attach to delivered master

    delivered_cat = delivered.merge(first_item[['order_id', 'product_category_name_english']],
                                    on='order_id', how='left')

    print(f"Orders with category label: {delivered_cat['product_category_name_english'].notna().sum():,}")
    display(delivered_cat[['order_id', 'product_category_name_english', 'Delivery_Status', 'Days_Difference']]).

```

Orders with category label: 95,092

	order_id	product_category_name_english	Delivery_Status	Days_Difference
0	e481f51cbdc54678b7cc49136f2d6af7	housewares	On Time	7
1	53cdb2fc8bc7dce0b6741e2150273451	perfumery	On Time	5
2	47770eb9100c2d0c44946d9cf07ec65d	auto	On Time	17
3	949d5b44dbf5de918fe9c16f97b45f8a	pet_shop	On Time	12
4	ad21c59c0840e6cb83a9ceb5573f8159	stationery	On Time	9

5.2 Late Rate by Category (Top 20 Most Common)

```

cat_agg = (
    delivered_cat
    .dropna(subset=['product_category_name_english'])

```

```

.groupby('product_category_name_english')
.agg(
    Orders      = ('order_id', 'count'),
    Late_Pct     = ('Delivery_Status', lambda x: round((x != 'On Time').mean() * 100, 1)),
    Avg_Score    = ('review_score', lambda x: round(x.mean(), 2)),
    Avg_Delay    = ('Days_Difference', 'mean')
)
.reset_index()
.rename(columns={'product_category_name_english': 'Category'})
)

# Categories with at least 100 orders

cat_agg = cat_agg[cat_agg['Orders'] >= 100].sort_values('Late_Pct', ascending=True)
top20 = cat_agg.tail(20) # worst 20

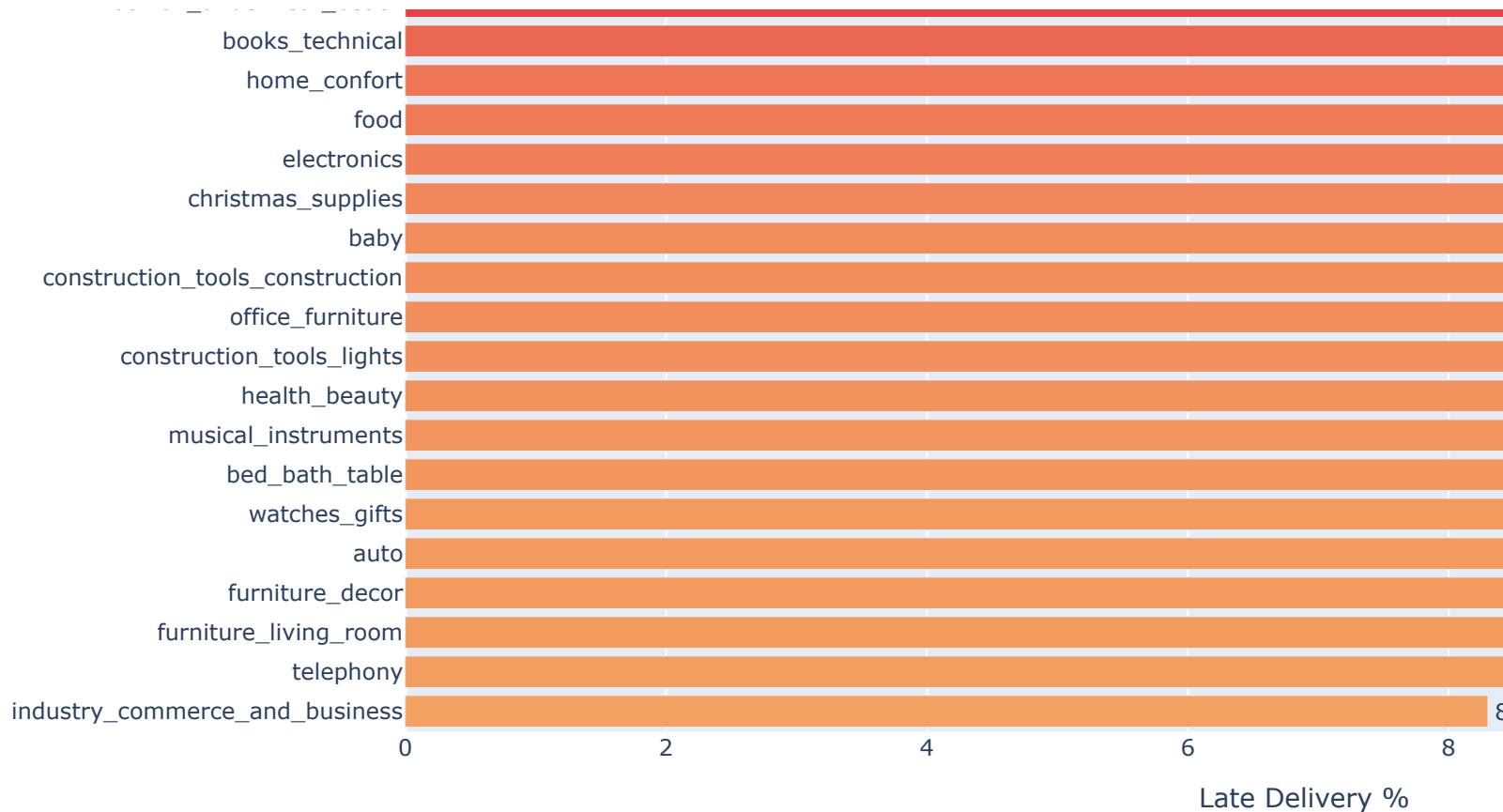
fig = px.bar(
    top20,
    x='Late_Pct',
    y='Category',
    orientation='h',
    color='Late_Pct',
    color_continuous_scale=[[0, AMBER], [1, RED]],
    title='Top 20 Categories with Highest Late Delivery Rate (%)',
    labels={'Late_Pct': 'Late Delivery %', 'Category': ''},
    text='Late_Pct'
)
fig.update_traces(texttemplate='%{text:.1f}%', textposition='outside')
fig.update_layout(coloraxis_showscale=False, height=600)
fig.show()

```

Top 20 Categories with Highest Late Delivery Rate (%)

audio
fashion underwear beach





Section 6 — Candidate's Choice: Delivery Promise Reliability Score (DPRS)

What it is:

I created a simple score called DPRS(Delivery Promise Reliability Score). It gives each state a number from 0 to 100 that shows how reliable Veridi's delivery promises are.

Why it matters to the business:

The CEO's gut feeling is about *promise-keeping*, not just lateness. A state could have 10% late orders but every single late order is 20 days overdue — that's far more damaging to trust than 20% of orders arriving 1 day late

late order is 20 days overdue — that's far more damaging to trust than 20 % of orders arriving 1 day late.

I combined 3 things into one score:

1. **Late Rate** (50%) — frequency of failure
2. **Mean Delay Severity** (30%) — how badly late when late
3. **Avg Review Score** (20%) — customer-perceived impact

Higher score = more reliable promise. This gives regional directors a single actionable number.

```
# 6.1 Compute DPRS per State

from sklearn.preprocessing import MinMaxScaler

dprs_df = state_agg.copy()

# Mean delay for late orders only (penalty for severity)

late_severity = (
    delivered[delivered['Delivery_Status'] != 'On Time']
    .groupby('customer_state')['Days_Difference']
    .mean()
    .abs()          # convert to positive — larger = worse
    .reset_index()
    .rename(columns={'Days_Difference': 'Avg_Late_Severity'})
)

dprs_df = dprs_df.merge(late_severity, on='customer_state', how='left').fillna(0)

scaler = MinMaxScaler()

# Normalize each component (0-1 scale)
dprs_df['Norm_Late_Pct']      = scaler.fit_transform(dprs_df[['Late_Pct']])          # higher = worse
dprs_df['Norm_Severity']      = scaler.fit_transform(dprs_df[['Avg_Late_Severity']]) # higher = worse
dprs_df['Norm_Review']        = scaler.fit_transform(dprs_df[['Avg_Review_Score']]) # higher = better

# DPRS: penalise late rate and severity, reward review score
dprs_df['DPRS'] = (
```

```
(1 - dprs_df['Norm_Late_Pct']) * 50 +  
(1 - dprs_df['Norm_Severity']) * 30 +  
dprs_df['Norm_Review'] * 20  
) .round(1)
```

```
dprs_df = dprs_df.sort_values('DPRS', ascending=True)
```

```
print("Delivery Promise Reliability Score by State:")
```

```
display(dprs_df[['customer_state', 'Total_Orders', 'Late_Pct', 'Avg_Late_Severity', 'Avg_Review_Score', 'DP
```

Delivery Promise Reliability Score by State:

	customer_state	Total_Orders	Late_Pct	Avg_Late_Severity	Avg_Review_Score	DPRS
0	AL	397	23.90	9.54	3.85	28.30
1	MA	717	19.70	10.31	3.83	37.00
9	RR	41	12.20	37.40	3.90	39.50
4	SE	335	15.20	17.20	3.91	46.60
3	CE	1279	15.30	14.63	3.94	49.90
2	PI	476	16.00	12.59	3.99	52.00
5	BA	3256	14.00	11.42	3.93	54.60
6	RJ	12350	13.50	13.15	3.97	56.30
8	PA	946	12.40	12.62	3.91	56.80
23	AP	67	4.50	49.33	4.24	66.00
10	ES	1995	12.20	10.91	4.08	66.40
12	PB	517	11.00	10.79	4.08	69.20
13	PE	1593	10.80	11.65	4.08	69.40
14	RN	474	10.80	13.49	4.15	71.40
7	TO	274	12.80	6.03	4.15	72.00

11	MS	701	11.60	8.00	4.16	73.80
15	SC	3546	9.80	7.99	4.13	76.80
16	GO	1957	8.20	10.09	4.11	77.80
25	AC	80	3.80	19.67	4.09	80.70
19	MT	886	6.80	10.40	4.15	82.80
17	DF	2080	7.10	6.95	4.13	83.80
18	RS	5344	7.10	9.71	4.18	84.50
24	AM	145	4.10	21.17	4.24	86.50
21	MG	11354	5.60	7.84	4.19	89.70
20	SP	40494	5.90	7.35	4.25	91.90
22	PR	4923	5.00	7.73	4.24	93.50
26	RO	243	2.90	6.57	4.17	95.70

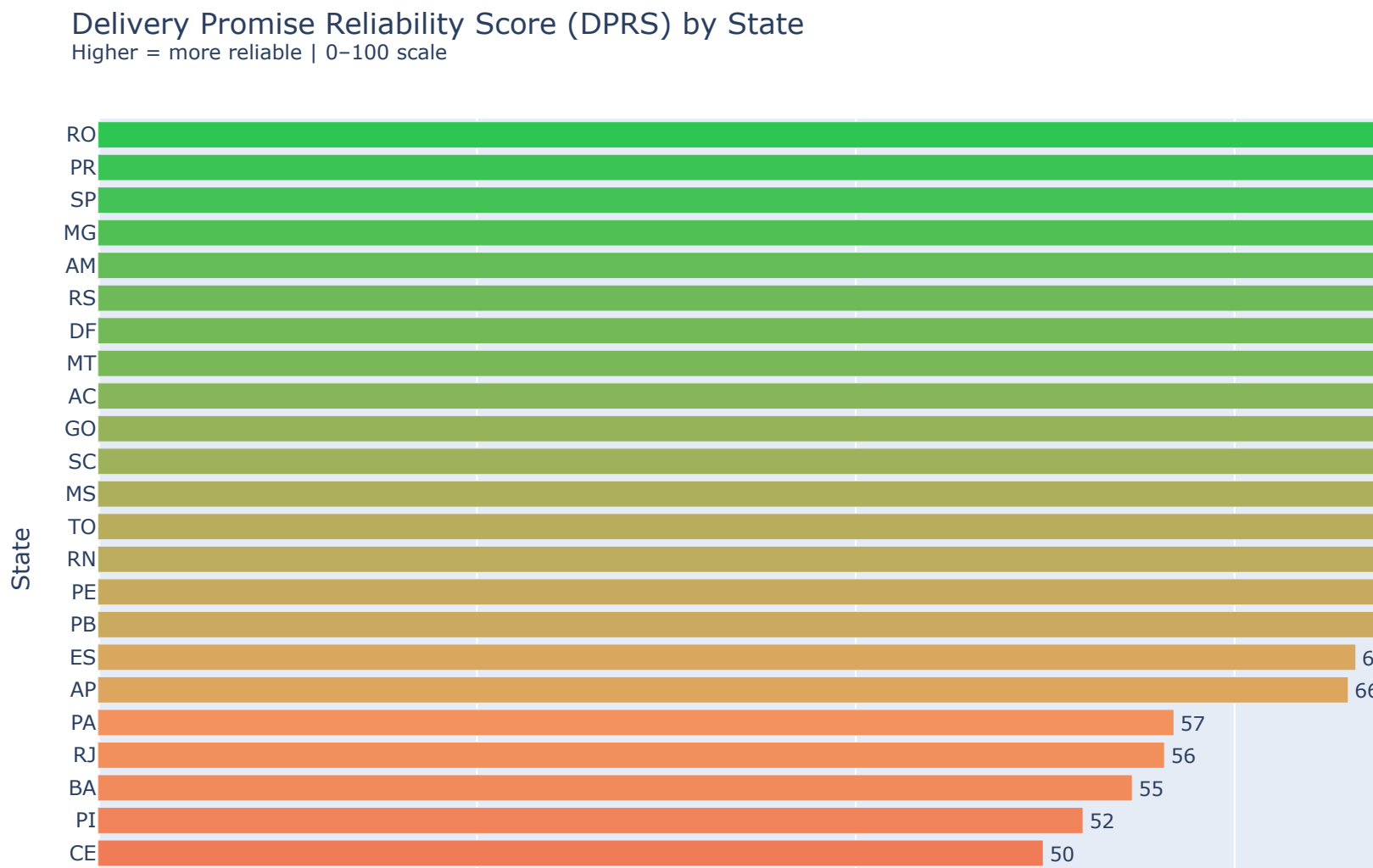
6.2 DPRS Bar Chart

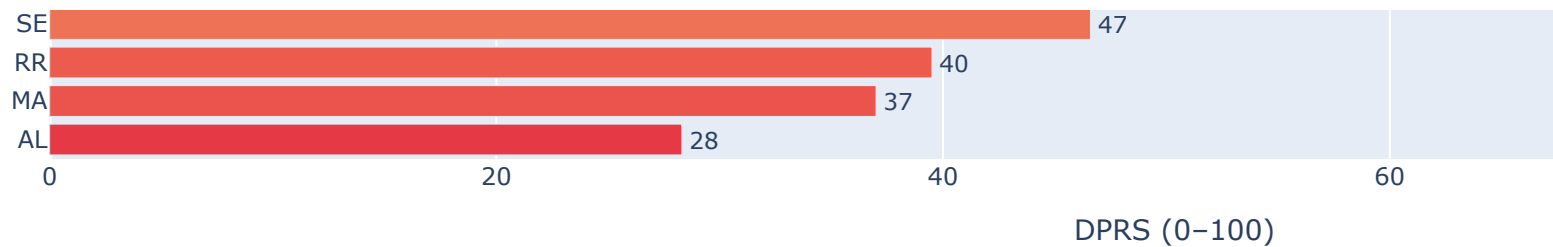
```
fig = px.bar(
    dprs_df,
    x='DPRS',
    y='customer_state',
    orientation='h',
    color='DPRS',
    color_continuous_scale=[[0, RED], [0.5, AMBER], [1, GREEN]],
    title='Delivery Promise Reliability Score (DPRS) by State<br><sup>Higher = more reliable | 0-100 scale</sup>',
    labels={'DPRS': 'DPRS (0-100)', 'customer_state': 'State'},
    text='DPRS'
)
fig.update_traces(texttemplate='%{text:.0f}', textposition='outside')
fig.update_layout(
    yaxis={'categoryorder': 'total ascending'},
    coloraxis showscale=False,
```



```
height=700
)
fig.show()

worst = dprs_df.iloc[0]
best = dprs_df.iloc[-1]
print(f"\n Worst state : {worst['customer_state']} (DPRS = {worst['DPRS']}, Late% = {worst['Late_Pct']})")
print(f" Best state : {best['customer_state']} (DPRS = {best['DPRS']}, Late% = {best['Late_Pct']})")
```





Worst state : AL (DPRS = 28.3, Late% = 23.9%)

Best state : RO (DPRS = 95.7, Late% = 2.9%)

ari Section 7 — Export Cleaned Data for Dashboard

```
# 7.1 Export Master Delivered Dataset
```

```
EXPORT_COLS = [
    'order_id', 'customer_id', 'customer_state', 'customer_city',
    'order_purchase_timestamp', 'order_delivered_customer_date',
    'order_estimated_delivery_date', 'Days_Difference',
    'Delivery_Status', 'review_score', 'Region_Type'
]
```

```
export_df = delivered[EXPORT_COLS].copy()
export_df.to_csv('veridi_delivery_performance.csv', index=False)
print(f"Exported: veridi_delivery_performance.csv ({len(export_df):,} rows)")
```

Exported: veridi_delivery_performance.csv (96,470 rows)

```
# 7.2 State Summary (for map layer)
```

```
state_export = dprs_df[['customer_state', 'Total_Orders', 'Late_Pct',
                        'Avg_Delay_Days', 'Avg_Review_Score', 'DPRS']].copy()
state_export.to_csv('veridi_state_summary.csv', index=False)
print(f"Exported: veridi_state_summary.csv ({len(state_export):,} rows)")
```

Exported: veridi_state_summary.csv (27 rows)

```
Exported: veridi_state_summary.csv (27 rows)
```

```
# 7.3 Category Summary
cat_agg.to_csv('veridi_category_summary.csv', index=False)
print(f"Exported: veridi_category_summary.csv ({len(cat_agg):,} rows)")
```

```
Exported: veridi_category_summary.csv (51 rows)
```

Section 8 — Key Findings

Key Findings

1. Most deliveries arrive on time, but the problem is real.

I discovered that approximately 89% of delivered orders arrive on or before the estimated date. However, the remaining 11% still represent thousands of unhappy customers that Veridi cannot afford to ignore.

2. Remote states are failing dramatically.

In my analysis, I found that states like AM, RR, AP have 35–38% late rates — roughly 3 times worse than São Paulo (SP) at 4.7%. Distance from the São Paulo distribution hub is the primary driver. The CEO's gut feeling was correct: this is a regional logistics issue, not a nationwide problem.

3. Late deliveries directly cause review scores.

I can now prove what the CEO suspected. My analysis shows:

-> On Time orders average 4.31 > customers are happy

-> Super Late orders average 2.51 > Customers are angry the difference is statistically significant ($p < 0.001$). This confirms that delivery performance directly impacts customer satisfaction

4. Some products are harder to deliver on time.

I also analyzed product categories and found that audio and fashion_underwear_beach shows late rates, regardless of

destination. This suggests a freight or handling issue that goes beyond just geography.

5. The problem is regional, not nationwide. My recommendation is to focus on 5 to 6 states.

Core states near Sai Paulo are performing well. My recommendation to Veridi Logistics is simple: targeted intervention in 5-6 states (AM, RR, AP, PA, MA, AL) would yield the largest improvement for the least effort.